05 Pseudobulk differential expression

- Raw, non-integrated counts are grouped by per donor x sequencing batch x genotype x cell type into pseudobulk units.
- Units are tested with limma-voom: TMM normalization, `filterByExpr`, robust empirical Bayes and BH correction.
- Two contrasts are tested in excitatory neurons, inhibitory neurons and oligodendrocytes:

Contrast	Numerator / reference	Population	Model
`case_het_vs_case_hom-ref`	het / hom-ref	PIK3CA cases, batch 2023 only	`~ test_fac`
`case_hom-ref_vs_control_hom-ref`	case / control	hom-ref nuclei, both batches	`~ batch_fac + test_fac`

- Sequencing batch is used as a fixed covariate where applicable -Donor is used as a random effect and donor correlation (`duplicateCorrelation`) is estimated for the case-vs-control contrast, when a donor contributes more than one pseudobulk.

```
suppressPackageStartupMessages({
  library(SingleCellExperiment)
  library(Matrix)
  library(edgeR)
  library(limma)
})

source(file.path("..", "scripts", "helpers.R"))
paths <- load_paths()
```

Parameters

```
required_cell_types <- c("Excitatory_neurons", "Inhibitory_neurons", "Oligodendrocytes")
unresolved_labels   <- c("Neuron_unresolved", "NonNeuron_unresolved", "Unk", "Unknown")

pb_diagnosis        <- "PIK3CA"
exclude_donor_ids   <- c("mcd19035") # SLC35A2 case excluded
allowed_genotypes   <- c("hom-ref", "het", "high-var")
control_genotype    <- "hom-ref"

min_count_gene       <- 20
min_total_count_gene <- 50
fdr_threshold <- 0.05
lfc_threshold <- 0.3

analysis_batch_map <- c("2023" = "1", "2024" = "2")
contrasts <- list(
  list(
    id = "case_het_vs_case_hom-ref",
    numerator = "het",
```

```
    reference = "hom-ref",
    grouping_field = "genotype_canonical",
    batches = "1",
    population = list(case_ct_status = "case"),
    min_cells_pb = 10L,
    min_donors = 2L,
    min_pseudobulks_per_group = 2L,
    batch_inclusion = "never",
    duplicate_correlation = "never"
    ),
  list(
    id = "case_hom-ref_vs_control_hom-ref",
    numerator = "case",
    reference = "control",
    grouping_field = "case_ct_status",
    batches = c("1", "2"),
    population = list(genotype_canonical = "hom-ref"),
    min_cells_pb = 10L,
    min_donors = 2L,
    min_pseudobulks_per_group = 2L,
    batch_inclusion = "always",
    duplicate_correlation = "auto"
    )
  )
```

```
sce <- readRDS(file.path(paths$work_dir, "03_annotated.rds"))
sex <- read.delim(file.path(paths$work_dir, "02_sex_chromosome_flags.tsv"),
 ↪  stringsAsFactors = FALSE)
stopifnot(identical(sex$gene_id, rownames(sce)))
```

Pseudobulk aggregation

This analysis focuses on PIK3CA cases and controls. The SLC35A2 donor is excluded. Only resolved nuclei of the three required cell types are used, and genotype is assumed to be hom-ref for every control nucleus.

```
cd <- as.data.frame(colData(sce))
donor <- as.character(cd$donor_id); batch <- as.character(cd$Seq_batch)
geno  <- as.character(cd$genotype_canonical); ccs <- as.character(cd$case_ct_status)
label <- as.character(cd$final_celltype_label)

in_scope <- ((cd$genetic_diagnosis %in% pb_diagnosis & ccs %in% "case") | ccs %in%
 ↪  "control") &
            !(donor %in% exclude_donor_ids)

included <- in_scope & !(label %in% unresolved_labels) & label %in% required_cell_types

stopifnot(all(geno[included] %in% allowed_genotypes),
          all(geno[included & ccs == "control"] == control_genotype))

pb_id <- paste(donor, batch, geno, label, sep = "__")[included]

levels_sorted <- sort(unique(pb_id))

indicator <- Matrix::sparseMatrix(i = seq_along(pb_id), j = match(pb_id, levels_sorted),
 ↪  x = 1,
```

```
                                dims = c(length(pb_id), length(levels_sorted)))

pb_counts <- as.matrix(assay(sce, "counts")[, included, drop = FALSE] %*% indicator)

dimnames(pb_counts) <- list(rownames(sce), levels_sorted)

first <- match(levels_sorted, pb_id)
inc <- cd[included, ]

pb_meta <- data.frame(pseudobulk_id = levels_sorted, donor_id = inc$donor_id[first],
                      Seq_batch = as.character(inc$Seq_batch[first]),
                      genotype_canonical = inc$genotype_canonical[first],
                      case_ct_status = inc$case_ct_status[first],
                      final_celltype_label = inc$final_celltype_label[first],
                      n_cells = as.integer(table(pb_id)[levels_sorted]),
                      stringsAsFactors = FALSE)
c(n_included_cells = sum(included), n_pseudobulks = ncol(pb_counts))
```

```
## n_included_cells    n_pseudobulks
##              453               35
```

```
# Sex-chromosome genes are excluded
pb_counts <- pb_counts[!sex$is_sex[match(rownames(pb_counts), sex$gene_id)], , drop =
 ↪  FALSE]
nrow(pb_counts)
```

```
## [1] 18399
```

Design checks and limma-voom fit

```
preflight <- function(meta, cs) {
  meta <- meta[meta$n_cells >= cs$min_cells_pb, , drop = FALSE]
  grp <- meta[[cs$grouping_field]]
  num <- meta[grp %in% cs$numerator, ]
  ref <- meta[grp %in% cs$reference, ]

  blocker <- if (nrow(num) == 0L || nrow(ref) == 0L) {
    "numerator/reference group not both present after min-cell filtering"
  } else if (length(unique(num$donor_id)) < cs$min_donors ||
             length(unique(ref$donor_id)) < cs$min_donors) {
    paste0("fewer than ", cs$min_donors, " donors per group")
  } else if (nrow(num) < cs$min_pseudobulks_per_group ||
             nrow(ref) < cs$min_pseudobulks_per_group) {
    paste0("fewer than ", cs$min_pseudobulks_per_group, " pseudobulks per group")
  } else NA_character_

  list(meta = meta, blocker = blocker,
       summary = data.frame(contrast_id = cs$id,
                            donors_numerator = length(unique(num$donor_id)),
                            donors_reference = length(unique(ref$donor_id)),
                            pseudobulks_numerator = nrow(num), pseudobulks_reference =
                             ↪  nrow(ref),
                            estimable = is.na(blocker), blocker = blocker))
}
```

```r
fit_de <- function(counts, meta, cs) {
  meta$group_label <- as.character(meta[[cs$grouping_field]])
  meta$test_fac <- stats::relevel(factor(meta$group_label), ref = cs$reference)

  y <- edgeR::DGEList(counts = counts)
  y <- edgeR::calcNormFactors(y, method = "TMM")
  keep <- edgeR::filterByExpr(y, group = meta$group_label, min.count = min_count_gene,
                              min.total.count = min_total_count_gene)
  y <- y[keep, , keep.lib.sizes = FALSE]

  add_batch <- FALSE

  if (cs$batch_inclusion != "never" && length(unique(meta$Seq_batch)) > 1L) {
    d_try <- stats::model.matrix(~ factor(Seq_batch) + test_fac, data = meta)
    add_batch <- qr(d_try)$rank == ncol(d_try)

    if (cs$batch_inclusion == "always" && !add_batch) stop("batch design not full rank")
  }
  meta$batch_fac <- factor(meta$Seq_batch)

  design <- if (add_batch) stats::model.matrix(~ batch_fac + test_fac, data = meta) else
    stats::model.matrix(~ test_fac, data = meta)
  colnames(design) <- make.names(colnames(design))
  stopifnot(qr(design)$rank == ncol(design))

  block <- meta$donor_id
  can_block <- cs$duplicate_correlation != "never" && length(unique(block)) >= 3L &&
               any(duplicated(block))

  if (can_block) {
    v1 <- limma::voom(y, design)
    cor1 <- tryCatch(limma::duplicateCorrelation(v1, design, block = block), error =
↪   function(e) NULL)
    can_block <- !is.null(cor1) && is.finite(cor1$consensus)
  }

  if (can_block) {
    v <- limma::voom(y, design, block = block, correlation = cor1$consensus)
    cor2 <- tryCatch(limma::duplicateCorrelation(v, design, block = block), error =
↪   function(e) NULL)

    rho <- if (!is.null(cor2) && is.finite(cor2$consensus)) cor2$consensus else
↪   cor1$consensus
    fit <- limma::lmFit(v, design, block = block, correlation = rho)
  } else {
    v <- limma::voom(y, design)
    fit <- limma::lmFit(v, design)
  }

  fit <- limma::eBayes(fit, robust = TRUE)
  coef <- make.names(paste0("test_fac", cs$numerator))
  tt <- limma::topTable(fit, coef = coef, n = Inf, adjust.method = "BH", sort.by =
↪   "none")
```

```r
  de <- data.frame(gene_id = rownames(tt), tt, row.names = NULL, stringsAsFactors =
 ↪  FALSE)
  de$contrast_id <- cs$id; de$numerator <- cs$numerator; de$reference <- cs$reference
  de$sig <- is.finite(de$adj.P.Val) & is.finite(de$logFC) &
            de$adj.P.Val < fdr_threshold & abs(de$logFC) >= lfc_threshold

  list(de = de, design = design,
       diagnostics = data.frame(contrast_id = cs$id, tested_coefficient = coef,
                                duplicate_correlation_used = can_block,
                                batch_term_included = add_batch,
                                model_formula = if (add_batch) "~ batch_fac + test_fac"
                                 ↪  else "~ test_fac",
                                n_genes_before_filter = nrow(counts),
                                 ↪  n_genes_after_filter = sum(keep)))
}
```

```r
out_dir <- file.path(paths$work_dir, "05_differential_expression")

preflight_rows <- list()
for (ct in required_cell_types) {
  meta_ct <- pb_meta[pb_meta$final_celltype_label == ct, , drop = FALSE]

  for (cs in contrasts) {
    batches_raw <- names(analysis_batch_map)[analysis_batch_map %in% cs$batches]
    keep <- meta_ct$Seq_batch %in% batches_raw

    for (f in names(cs$population)) keep <- keep & meta_ct[[f]] == cs$population[[f]]
    keep <- keep & meta_ct[[cs$grouping_field]] %in% c(cs$numerator, cs$reference)
    pf <- preflight(meta_ct[keep, , drop = FALSE], cs)

    preflight_rows[[paste(ct, cs$id)]] <- cbind(pf$summary, cell_type = ct)

    if (!is.na(pf$blocker)) next
    res <- fit_de(pb_counts[, pf$meta$pseudobulk_id, drop = FALSE], pf$meta, cs)

    save_tsv(res$de, file.path(out_dir, ct, paste0(cs$id, "_toptable.tsv")))
    save_tsv(as.data.frame(res$design, check.names = FALSE),
             file.path(out_dir, ct, paste0(cs$id, "_design_matrix.tsv")))
    save_tsv(res$diagnostics, file.path(out_dir, ct, paste0(cs$id,
     ↪  "_model_diagnostics.tsv")))
  }
}

preflight_tab <- do.call(rbind, preflight_rows)
save_tsv(preflight_tab, file.path(out_dir, "preflight.tsv"))
preflight_tab
```

```
##                                                                    contrast_id
## Excitatory_neurons case_het_vs_case_hom-ref               case_het_vs_case_hom-ref
## Excitatory_neurons case_hom-ref_vs_control_hom-ref case_hom-ref_vs_control_hom-ref
## Inhibitory_neurons case_het_vs_case_hom-ref               case_het_vs_case_hom-ref
## Inhibitory_neurons case_hom-ref_vs_control_hom-ref case_hom-ref_vs_control_hom-ref
## Oligodendrocytes case_het_vs_case_hom-ref                 case_het_vs_case_hom-ref
## Oligodendrocytes case_hom-ref_vs_control_hom-ref   case_hom-ref_vs_control_hom-ref
```

```
##                                                    donors_numerator
## Excitatory_neurons case_het_vs_case_hom-ref                       2
## Excitatory_neurons case_hom-ref_vs_control_hom-ref                2
## Inhibitory_neurons case_het_vs_case_hom-ref                       1
## Inhibitory_neurons case_hom-ref_vs_control_hom-ref                0
## Oligodendrocytes case_het_vs_case_hom-ref                         2
## Oligodendrocytes case_hom-ref_vs_control_hom-ref                  2
##                                                    donors_reference
## Excitatory_neurons case_het_vs_case_hom-ref                       2
## Excitatory_neurons case_hom-ref_vs_control_hom-ref                5
## Inhibitory_neurons case_het_vs_case_hom-ref                       0
## Inhibitory_neurons case_hom-ref_vs_control_hom-ref                3
## Oligodendrocytes case_het_vs_case_hom-ref                         2
## Oligodendrocytes case_hom-ref_vs_control_hom-ref                  2
##                                                    pseudobulks_numerator
## Excitatory_neurons case_het_vs_case_hom-ref                            2
## Excitatory_neurons case_hom-ref_vs_control_hom-ref                     2
## Inhibitory_neurons case_het_vs_case_hom-ref                            1
## Inhibitory_neurons case_hom-ref_vs_control_hom-ref                     0
## Oligodendrocytes case_het_vs_case_hom-ref                              2
## Oligodendrocytes case_hom-ref_vs_control_hom-ref                       2
##                                                    pseudobulks_reference
## Excitatory_neurons case_het_vs_case_hom-ref                            2
## Excitatory_neurons case_hom-ref_vs_control_hom-ref                     5
## Inhibitory_neurons case_het_vs_case_hom-ref                            0
## Inhibitory_neurons case_hom-ref_vs_control_hom-ref                     3
## Oligodendrocytes case_het_vs_case_hom-ref                              2
## Oligodendrocytes case_hom-ref_vs_control_hom-ref                       2
##                                                    estimable
## Excitatory_neurons case_het_vs_case_hom-ref             TRUE
## Excitatory_neurons case_hom-ref_vs_control_hom-ref      TRUE
## Inhibitory_neurons case_het_vs_case_hom-ref            FALSE
## Inhibitory_neurons case_hom-ref_vs_control_hom-ref     FALSE
## Oligodendrocytes case_het_vs_case_hom-ref               TRUE
## Oligodendrocytes case_hom-ref_vs_control_hom-ref        TRUE
##
 ↪  blocker
## Excitatory_neurons case_het_vs_case_hom-ref
 ↪  <NA>
## Excitatory_neurons case_hom-ref_vs_control_hom-ref
 ↪  <NA>
## Inhibitory_neurons case_het_vs_case_hom-ref        numerator/reference group not both
 ↪  present after min-cell filtering
## Inhibitory_neurons case_hom-ref_vs_control_hom-ref numerator/reference group not both
 ↪  present after min-cell filtering
## Oligodendrocytes case_het_vs_case_hom-ref
 ↪  <NA>
## Oligodendrocytes case_hom-ref_vs_control_hom-ref
 ↪  <NA>
##                                                              cell_type
## Excitatory_neurons case_het_vs_case_hom-ref        Excitatory_neurons
## Excitatory_neurons case_hom-ref_vs_control_hom-ref Excitatory_neurons
## Inhibitory_neurons case_het_vs_case_hom-ref        Inhibitory_neurons
## Inhibitory_neurons case_hom-ref_vs_control_hom-ref Inhibitory_neurons
## Oligodendrocytes case_het_vs_case_hom-ref            Oligodendrocytes
```

```
## Oligodendrocytes case_hom-ref_vs_control_hom-ref     Oligodendrocytes
```

Check against uploaded results

```
dep_dir <- file.path(paths$zenodo_root, "results", "differential_expression")

for (f in list.files(out_dir, "_toptable\\.tsv$", recursive = TRUE)) {
  a <- read.delim(file.path(out_dir, f)); b <- read.delim(file.path(dep_dir, f))
  stopifnot(identical(a$gene_id, b$gene_id),
            isTRUE(all.equal(a$t, b$t, tolerance = 1e-8)),
            isTRUE(all.equal(a$adj.P.Val, b$adj.P.Val, tolerance = 1e-8)))
}
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.5.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.
 ↪  framework/Versions/A/libBLAS.dylib
## LAPACK:
 ↪  /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
 ↪  LAPACK version 3.12.1
##
## locale:
## [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats4    stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] edgeR_4.8.2                 limma_3.66.0
##  [3] igraph_2.2.1                scater_1.38.0
##  [5] ggplot2_4.0.1               scran_1.38.0
##  [7] Matrix_1.7-4                scuttle_1.20.0
##  [9] SingleCellExperiment_1.32.0 SummarizedExperiment_1.40.0
## [11] Biobase_2.70.0              GenomicRanges_1.62.1
## [13] Seqinfo_1.0.0               IRanges_2.44.0
## [15] S4Vectors_0.48.0            BiocGenerics_0.56.0
## [17] generics_0.1.4              MatrixGenerics_1.22.0
## [19] matrixStats_1.5.0
##
## loaded via a namespace (and not attached):
##  [1] beeswarm_0.4.0      gtable_0.3.6        xfun_0.55
##  [4] ggrepel_0.9.6       lattice_0.22-7      vctrs_0.7.1
##  [7] tools_4.5.2         parallel_4.5.2      tibble_3.3.1
## [10] cluster_2.1.8.2     pkgconfig_2.0.3     BiocNeighbors_2.4.0
## [13] RColorBrewer_1.1-3  S7_0.2.1            dqrng_0.4.1
## [16] lifecycle_1.0.5     compiler_4.5.2      farver_2.1.2
## [19] statmod_1.5.1       tinytex_0.58        bluster_1.20.0
```

```
## [22] codetools_0.2-20    vipor_0.4.7         htmltools_0.5.9
## [25] yaml_2.3.12         pillar_1.11.1       BiocParallel_1.44.0
## [28] DelayedArray_0.36.0 viridis_0.6.5       abind_1.4-8
## [31] tidyselect_1.2.1    rsvd_1.0.5          locfit_1.5-9.12
## [34] metapod_1.18.0      digest_0.6.39       BiocSingular_1.26.0
## [37] dplyr_1.2.0         fastmap_1.2.0       grid_4.5.2
## [40] cli_3.6.5           SparseArray_1.10.8  magrittr_2.0.4
## [43] S4Arrays_1.10.1     withr_3.0.2         scales_1.4.0
## [46] ggbeeswarm_0.7.3    rmarkdown_2.30      XVector_0.50.0
## [49] otel_0.2.0          gridExtra_2.3       ScaledMatrix_1.18.0
## [52] beachmat_2.26.0     evaluate_1.0.5      knitr_1.51
## [55] viridisLite_0.4.2   irlba_2.3.7         rlang_1.1.7
## [58] Rcpp_1.1.1          glue_1.8.0          R6_2.6.1
```